arXiv:2610.34126v1 [cs.RO] 25 Oct 2026

HumanoidNDM: Closing the Training-Deployment Gap of Motion Tracking via Humanoid Neural Dynamics Model

Chao Chen[1,3,*] **Zhenrong Wang**[2,3,4,*] **Jilong Wang**[3] **Yusen Feng**[1,3]
Jiangran Lyu[1,3] **Li Yi**[3,5,†] **He Wang**[1,3,†]
[1]CFCS, School of Computer Science, Peking University
[2]Institute of Automation, Chinese Academy of Sciences
[3]Galbot [4]Beijing Academy of Artificial Intelligence [5]Tsinghua University
[*]Equal contribution. [†]Corresponding authors.

Project Page: https://pku-epic.github.io/HumanoidNDM

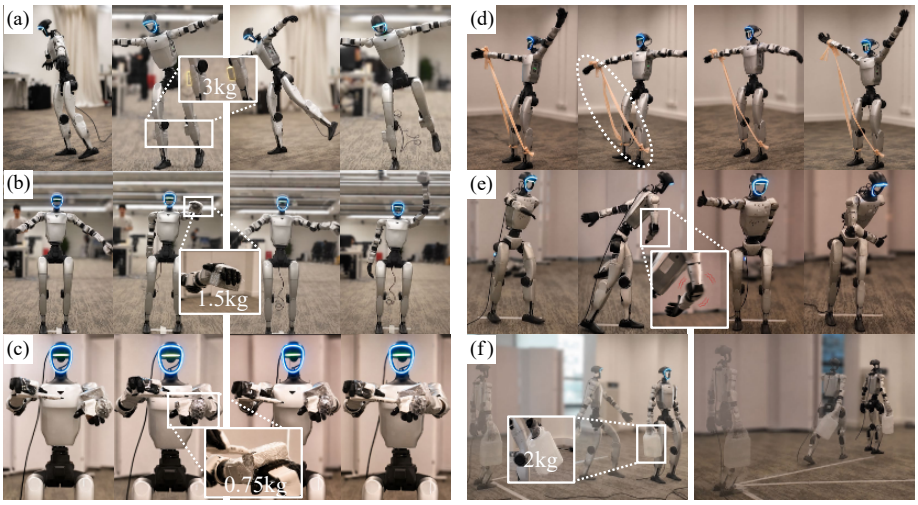


Figure 1: Comparison of direct transfer (left two frames) and fine-tuned performance (right two frames): (a) **Swallow Balance**: 1.5 kg weights on each calf; (b) **Arm Swing**: 1.5 kg weight on the left hand; (c) **Pen Capping**: 0.75 kg load on the left hand; (d) **Lateral Raise**: two elastic bands attached to wrists; (e) **Bowing**: damaged arm motor; and (f) **Walking**: 2 kg water bucket in hand.

Abstract: Humanoid robots have recently achieved remarkable success in motion tracking. However, during real-world deployment, robots inevitably encounter dynamics gaps—arising from either extra physical disturbances or inaccurate simulation modeling—resulting in a significant training-deployment gap. Therefore, effectively leveraging real-world data to fine-tune policies represents the crucial "last mile" for robust deployment. To bridge this gap, we propose HumanoidNDM, an online fine-tuning framework for humanoid motion tracking driven by a real-world-trained neural dynamics model. First, in the data collection stage, we introduce a low-frequency reference perturbation method with an adaptive intensity mechanism, balancing deployment feasibility and exploration efficiency. Second, for the model training stage, we train a robust history-based, proprioceptive neural dynamics model in Euclidean space by incorporating differentiable forward kinematics, eliminating the reliance on costly motion capture systems. Finally, in the policy fine-tuning stage, we directly fine-tune the policy via back-propagated model gradients, achieving high data efficiency. Through physical validation on the Unitree G1 robot, we demonstrate that HumanoidNDM enables rapid real-world adaptation to various perturbations, representing the first online, MoCap-free fine-tuning framework for humanoid tracking policies.

Keywords: Sim-to-Real Transfer, Humanoid Robots, Motion Tracking

10th Conference on Robot Learning (CoRL 2026), Austin TX, USA.

1 Introduction

Advancements in reinforcement learning [1, 2] and motors [3, 4] have enabled remarkable success in robot control [5, 6], especially in humanoid motion tracking [7, 8, 9]. However, most methods still rely on zero-shot real-world deployment of simulation-trained policies [7, 8, 9]. We argue that this paradigm has critical limitations. First, during real-world deployment, robots inevitably encounter training-deployment gaps, such as extra physical disturbances or inaccurate simulation modeling, which severely degrade policy performance. Second, when real-world performance is poor, adjusting simulation training parameters is highly inefficient as it relies on black-box tuning. Therefore, effectively leveraging real-world data to fine-tune simulation-trained policies represents the crucial "last mile" for achieving robust practical deployment.

In this paper, using tracking as a representative task, we focus on the real-world adaptation of humanoid tracking policies. Inspired by how humans continuously refine their physical skills, we propose **HumanoidNDM**, which abstracts the online learning paradigm into three stages: **Data Collection**, **Model Training**, and **Policy Fine-tuning**.

However, direct hardware deployment introduces stage-specific challenges across these three stages. First, during data collection, existing approaches often either employ exogenous noise injection (e.g. add Gaussian noise to action), which leads to violent joint jitter and whole-body shaking on physical robots [10, 11], or depend solely on environmental stochasticity for passive exploration [12], which is highly detrimental to subsequent policy fine-tuning. As a result, existing methods fail to achieve a balance between deployment feasibility and exploration effectiveness. Second, in model training, learning whole-body dynamics relies heavily on privileged information obtained from external MoCap systems [12], which is costly and hard to deploy. Lastly, during policy training, achieving efficient and stable fine-tuning under limited online data remains a problem. HumanoidNDM resolves these bottlenecks step by step.

In the **Data Collection** stage, we propose **F**requency-domain **E**xploration via **N**eighborhood **C**overage **E**stimation (**FENCE**). To strike a balance between deployment feasibility and exploration effectiveness, FENCE injects low-frequency noise into the reference motions, leveraging inherent policy robustness for smoother and more stable exploration. We further introduce an adaptive mechanism that dynamically scales the perturbation intensity based on policy neighborhood coverage to enhance exploration efficiency.

In the **Model Training** stage, while optimizing models in Euclidean space has become a consensus in existing works [13, 14], doing so for humanoid robots typically relies on privileged MoCap data. To bypass this, we extend traditional single-frame models with historical observations and differentiable forward kinematics (FK), enabling direct optimization in Euclidean space via purely proprioceptive data, which significantly improves the model's reliability and robustness.

Finally, in the **Policy Fine-tuning** stage, we adopt a supervised learning paradigm that updates the policy via model gradients [15, 16, 17, 18]. Unlike prior methods constrained by privileged information and short rollouts that cause greedy updates, our proprioceptive model supports training over longer rollout horizons, thereby achieving efficient optimization across the entire trajectory.

Simulation experiments show HumanoidNDM achieves significant improvement under various training-deployment gaps, surpassing baselines across various metrics. Real-world experiments demonstrate that HumanoidNDM achieves efficient adaptation against challenging disturbances.

The main contributions of this paper are summarized as follows:

- We propose FENCE, a deployment-feasible real-world exploration method using low-frequency reference perturbations, along with an adaptive intensity mechanism.
- We propose methods for training MoCap-free robust neural dynamics models, and subsequently leverage the model's differentiability to enable efficient policy fine-tuning.
- We deploy the first online fine-tuning framework for humanoid tracking policies, validating its ability to adapt to challenging disturbances.

2 Related Work

2.1 Learning Robotics Control Policies with Real-World Data

Existing approaches for bridging the training-deployment gap can be broadly divided into simulator-based adaptation and real-world dynamics learning. Simulator-based adaptation methods align simulation with reality through system identification [19, 20], transition compensation [12, 21], or trajectory-level alignment [22]. However, these approaches typically carry high OOD risks, and suffer from slow training speeds, which precludes online policy updates.

In contrast, real-world training directly leverages deployment data collected from physical robots to update policies. Model-free RL directly applies off-policy RL methods in real world [23, 24, 25, 26, 27, 28], but remains unstable under limited real-world data and complex humanoid dynamics [29, 30]. Model-based methods improve data efficiency by learning neural dynamics from real-world interactions, either for value-based policy updates [31, 32, 33, 34] or for supervised policy optimization through analytical model gradients [35]. HumanoidNDM adopts the latter paradigm for its data efficiency in real-world online adaptation.

2.2 Real-world Exploration in Legged Robots

Exploration is essential for policy improvement [36, 37, 38], yet simulation-oriented strategies such as uncorrelated Gaussian action noise can induce unsafe high-frequency torques and destructive vibrations on hardware [10, 11]. To avoid this, some methods rely on passive environmental stochasticity [33, 12], but this limits state-action coverage and further optimization. Some other methods try to introduce temporally correlated or trajectory-level perturbations [39, 40, 41, 42] for smooth exploration, but remain tuning-sensitive and weakly validated on real robots. While guided exploration methods utilizing simulation priors [43], Fisher information [20], or adaptive noise envelopes [44] attempt to realize targeted and controllable exploration, they still struggle with actual deployment feasibility on hardware.

To realize deployment-viable and efficient exploration, we propose FENCE, which injects structured low-frequency perturbations with an adaptive intensity into reference trajectories. Table 1 further compares FENCE with representative exploration strategies.

Table 1: Comparison of exploration strategies. "$\sim$" denotes partial or conditional capability.

Method	Deployment Feasibility	Policy-Relevant Coverage	Adaptive Intensity
Uncorrelated Action Noise [45, 46]	$\times$	✓	$\times$
Observation Perturbation [47, 48]	$\times$	$\times$	$\times$
Passive Dynamic Exploration [33, 12]	✓	$\times$	$\times$
Temporally Correlated Noise [39, 40]	$\sim$	✓	$\times$
Ours (FENCE)	✓	✓	✓

3 Preliminaries

The motion tracking task requires the robot to replicate a reference trajectory $\mathcal{T} = \{\hat{q}_t, \hat{\dot{q}}_t\}_{t=1}^{T}$, where $\hat{q}_t$ and $\hat{\dot{q}}_t$ denote the target joint positions (*qpos*) and velocities (*qvel*). At each step t, the locomotion policy $\pi(a_t|o_t)$ receives an observation o_t comprising the proprioceptive state (q_t, $\dot{q}_t$, gravity vector $\mathbf{g}_t$, and angular velocity $\boldsymbol{\omega}_t$), previous action a_{t-1}, and future reference states from $\mathcal{T}$. The policy outputs action $a_t \in [-1, 1]$. While π is agnostic to the training algorithm, our base policy is trained using PPO [1] in the highly parallelized MuJoCo-JAX (MJX) [49].

Policy output a_t is scaled by a factor c and added to the reference position to get position target q_t^{des}: $q_t^{des} = c \cdot a_t + \hat{q}_t$. A proportional-derivative (PD) controller then computes the joint torques τ_t:

$$\tau_t = K_p(q_t^{des} - q_t) - K_d\dot{q}_t$$

where K_p and K_d are the PD gains. Finally, τ_t is clipped to the hardware limits and executed.

4 Method

This section details HumanoidNDM, which abstracts online real-world adaptation into three stages. We then introduce its three core components: the deployment-viable data collection mechanism FENCE, MoCap-free world model learning, and efficient model-gradient-guided policy fine-tuning.

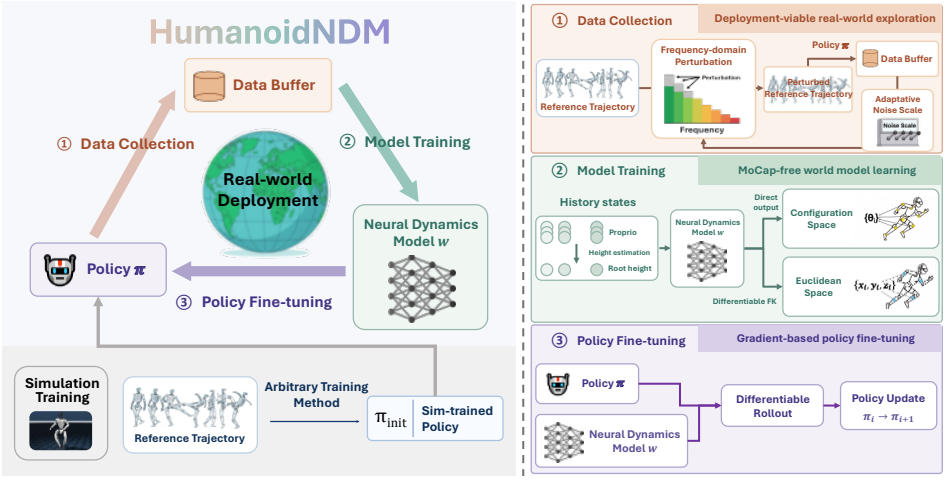


Figure 2: Pipeline of HumanoidNDM. Left: HumanoidNDM uses a simulation-trained policy to collect real-world data for online fine-tuning. Each iteration comprises three stages: **Data Collection**, **Model Training**, and **Policy Fine-tuning**. Right: The key components of each stage.

4.1 Deployment-viable Real-world Exploration with Frequency-Domain Perturbations

This section introduces FENCE, a data collection mechanism that combines frequency-domain perturbations on reference trajectories with dynamic intensity adjustment.

Frequency-Domain Perturbation on Reference Trajectory We propose a low-frequency perturbation method applied to the reference trajectory for real-world exploration inspired by human behavior. When exploring new environments, humans smoothly adjust desired poses (reference motion) instead of injecting high-frequency jitter. By adding low-frequency noise to reference trajectory, our method enables stable exploration that exploits the policy's closed-loop robustness.

Formally, let the reference joint position sequence be $\mathbf{Q} = [\hat{q}_1, \ldots, \hat{q}_T]^\top \in \mathbb{R}^{T \times D}$. We first apply the Discrete Fourier Transform to obtain the frequency-domain spectrum $\tilde{\mathbf{Q}}$:

$$\tilde{\mathbf{Q}}_k = \sum_{t=1}^{T} \hat{q}_t e^{-i2\pi k(t-1)/T}, \tag{1}$$

where $k \in \{0, 1, \ldots, T-1\}$ is the frequency index. To introduce smooth deviations, we construct a frequency-domain noise matrix $\mathbf{Z} \in \mathbb{C}^{T \times D}$, where $\mathbf{Z}_0$ is set to $\mathbf{0}$ to prevent posture collapse. For low-frequency components $0 < k \leq k_c$, we sample complex Gaussian noise $\mathbf{Z}_k \sim \mathcal{CN}(0, \sigma^2 \mathbf{I})$, where σ dictates the perturbation intensity, and set $Z_{T-k} = Z_k^*$ to preserve conjugate symmetry. All remaining frequency components are set to zero. The perturbed reference trajectory $\mathbf{Q}'$ is then recovered via the Inverse DFT:

$$\mathbf{Q}' = \mathcal{F}^{-1}(\tilde{\mathbf{Q}} + \mathbf{Z}). \tag{2}$$

Other reference states (e.g., joint velocities) are subsequently re-derived from $\mathbf{Q}'$ to maintain kinematic consistency. During rollouts, the policy π tracks $\mathbf{Q}'$ instead of $\mathbf{Q}$.

Adaptive Perturbation Intensity via Policy Neighborhood Coverage In model-based training, distribution shifts frequently lead to objective mismatch and model exploitation [50, 51], making it critical for data collection to cover the state-action subspace visited by the policy [52, 53]. To address

this, we introduce an adaptive perturbation intensity mechanism. To provide a quantitative criterion for adjustment, we extend traditional cell-based state space discretization [54] to our continuous control setting, defining the Policy Neighborhood Coverage C. Specifically, let $x_t = [s_t^\top, a_t^\top]^\top$ denote the concatenated state and action at step t, obtained by executing the latest policy without perturbation. Around each point x_t, we define a local neighborhood of radius R, uniformly partitioned into N_r radial bins and N_d directional bins. By aggregating the binary occupancy of the buffer $\mathcal{B}$ across all spatial bins along the entire trajectory, it is formulated as:

$$C = \frac{1}{TN_rN_d} \sum_{t=1}^{T-1} \sum_{j=1}^{N_r} \sum_{k=1}^{N_d} \mathbb{I}\left(\mathcal{B} \cap B_t^{(j,k)} \neq \emptyset\right), \tag{3}$$

where $B_t^{(j,k)}$ denotes the spatial bin at the j-th radial and k-th directional index. Intuitively, C measures the uniformity and density of exploration around the current policy. To implement online intensity adjustment, we discretize perturbation intensity into K candidate levels $\Sigma = \{\sigma_1, \ldots, \sigma_K\}$, and utilize Multi-Armed Bandit (MAB) to adaptively select the optimal perturbation level.

4.2 MoCap-Free Neural Dynamics Model Training with Dual-Space Objective

Although neural networks are commonly used to model environment dynamics [16, 55, 56], learning an accurate world model for humanoid motion tracking is challenging in a MoCap-free setting. During deployment, the robot can only access onboard proprioceptive signals, while external full-body pose measurements are unavailable. A straightforward solution is to optimize the dynamics model in configuration space. However, for humanoid robots, small configuration-space errors may cause large deviations in link positions, end-effector poses, and contact-related body configurations, which can severely degrade downstream policy fine-tuning [14, 13].

Following physics-based animation works [14, 13, 12], we introduce dual-space objectives into neural dynamics model training. The key is to preserve a proprioceptive-only interface while using differentiable Forward Kinematics (FK) for Euclidean-space supervision.

We define the proprioceptive state as $s_t = \{q_t, \dot{q}_t, \mathbf{g}_t, \boldsymbol{\omega}_t, h_t\}$, where the root height h_t is estimated from q_t and $\mathbf{g}_t$ assuming continuous ground contact. The world model w predicts the next state $\hat{s}_{t+1}$ by conditioning on a historical sequence of states and actions of length H:

$$\hat{s}_{t+1} = w(\mathbf{s}_{t-H+1:t}, \mathbf{a}_{t-H+1:t}). \tag{4}$$

This temporal context allows the model to implicitly infer recent dynamics. Furthermore, an ensemble framework with multiple model heads is trained via bootstrap sampling to enhance robustness.

To enable end-to-end dual-space optimization, we integrate a differentiable FK model that maps predicted configuration-space states into Euclidean space, yielding 6-DoF link poses in a local gravity-aligned frame. To ensure long-term predictive accuracy, we adopt an autoregressive rollout strategy [14, 13, 31, 32], recursively predicting future states $\hat{s}t + k$ over a window of length L_w by appending predictions to the input history, with the rollout loss $\mathcal{L}_w$ defined as

$$\mathcal{L}_w = \frac{1}{L_w} \sum_{k=1}^{L_w} \left(\mathcal{L}_{\text{config_err}}^{(t+k)} + \mathcal{L}_{\text{euclid_err}}^{(t+k)}\right). \tag{5}$$

The configuration-space error $\mathcal{L}_{\text{config_err}}$ comprises the L_2 distance for proprioceptive states ($q, \dot{q}, \mathbf{g}, \boldsymbol{\omega}$, and h), while the Euclidean-space error $\mathcal{L}_{\text{euclid_err}}$ penalizes the positional and rotational discrepancies of all links in the local gravity-aligned frame via the differentiable FK model.

4.3 Efficient and Stable Policy Fine-Tuning via Supervised Gradients

With the Euclidean-space neural dynamics model w established, we opt for direct optimization by backpropagating analytic gradients through the differentiable model [13, 14, 35]. This scheme is chosen for its superior data efficiency and faster convergence compared to model-based reinforcement learning [31, 32], which is crucial for limited real-world data.

Policy fine-tuning is analogous to the model training process. First, we adopt an autoregressive rollout strategy over a length L_π to avoid greedy updates. Furthermore, we use dual-space loss as the tracking objective, which significantly enhances the expressiveness of the tracked motions [57, 9, 8]. Incorporating standard regularization terms $\mathcal{L}_{\text{reg}}$, the policy fine-tuning loss $\mathcal{L}_\pi$ is formulated as:

$$\mathcal{L}_\pi = \frac{1}{L_\pi} \sum_{k=1}^{L_\pi} \left(\mathcal{L}_{\text{config_track}}^{(t+k)} + \mathcal{L}_{\text{euclid_track}}^{(t+k)} + \mathcal{L}_{\text{reg}}^{(t+k)} \right). \tag{6}$$

5 Experiments

In this section, we first present baseline comparisons and detailed ablation studies in simulation to validate our core algorithmic designs. Then we deploy HumanoidNDM on Unitree G1 to demonstrate its capability in closing the training-deployment gap under diverse physical disturbances.

5.1 Evaluation in Simulation

Experiments Setup The experiments are conducted in MuJoCo using a Unitree G1 model. We designed five distinct motions and trained a deployable initial policy. The five motions include planar and vertical motions, as well as actions emphasizing high dynamics and extreme balance. To systematically simulate training-deployment gap, we introduce three distinct types of disturbance. (1) **inertial variations**: adding mass and shifting the CoM of specific links. (2) **contact variations**: altering ground friction coefficients. (3) **actuation variations**: scaling the K_p of specific joints.

Baselines We compare HumanoidNDM with: (a) **PPO w/ Disturb.**, trained directly using PPO with the exact deployment disturbances; (b) **Direct Transfer**, where policies trained without disturbance are directly deployed to perturbed environments; (c) **ASAP** [12], which aligns sim-to-real dynamics using a delta action model trained on real-world transitions; and (d) **RWM** [32], another model-based method that differs from HumanoidNDM across all three stages. HumanoidNDM, ASAP, and RWM are trained using 200 trajectories collected from the deployment environment.

Metrics We evaluate tracking performance using 4 key metrics, all computed relative to the reference motion: (a) $E_{\text{link_pos}}$, the mean position error of all links in a local gravity-aligned frame; (b) $E_{\text{link_rot}}$, the mean rotation error of all links in a local gravity-aligned frame; (c) E_{qpos}, the tracking error of all joints; and (d) $E_{\text{key_qpos}}$, the joint error of the most expressive, task-specific joints.

To evaluate the stability of different exploration methods during data collection, we introduce two additional metrics: (a) S_{qpos}, the full-body qpos smoothness, which measures the joint jitter; and (b) $S_{\text{roll-pitch}}$, the roll-pitch smoothness, which evaluates the whole-body stability of the robot.

5.1.1 Comparison with Baselines

As shown in Table 2, HumanoidNDM demonstrates robust superiority, achieving the best performance in 40 out of 60 evaluated metrics. HumanoidNDM shows a clear advantage in the walk, squat, and swing motions, while its performance is comparable to other methods in the balance and stride motions. Since the training-deployment gap for balance and stride is smaller than that of the other three motions, the higher sample efficiency of HumanoidNDM compared to RWM is not fully highlighted, and ASAP's delta action model is also relatively easier to bridge the dynamic gap between the training and deployment environments.

Across all 15 settings, HumanoidNDM yields the most significant improvements in Euclidean space metrics. Normalized against the Direct Transfer baseline, our method reduces the $E_{\text{link_pos}}$ and $E_{\text{link_rot}}$ by 30% and 47%, respectively. In contrast, ASAP achieves reductions of 23% and 27%, while RWM reduces them by 9% and 35%. We point out that this improvement benefits from our explicit emphasis on Euclidean space optimization during model training and policy fine-tuning.

Finally, HumanoidNDM performs on par with the PPO w/ Disturb., which is directly trained using PPO in environments with the exact deployment disturbances. Matching this baseline demonstrates

Table 2: Comparison of HumanoidNDM and baselines across 5 random trials (mean±std).

Disturbance		Extra Weight				Scaled KP				Small Friction			
Trajectory	Method	$E_{\text{link_pos}}$	$E_{\text{link_rot}}$	E_{qpos}	$E_{\text{key_qpos}}$	$E_{\text{link_pos}}$	$E_{\text{link_rot}}$	E_{qpos}	$E_{\text{key_qpos}}$	$E_{\text{link_pos}}$	$E_{\text{link_rot}}$	E_{qpos}	$E_{\text{key_qpos}}$
Walk	PPO w/ Disturb.	3.11	0.28	2.01	3.83	2.74	0.28	1.70	1.94	2.45	0.24	1.46	1.70
	Direct Transfer	5.33±0.00	0.55±0.00	3.29±0.00	3.81±0.00	4.92±0.00	0.56±0.00	3.20±0.00	3.80±0.00	4.64±0.00	0.49±0.00	3.06±0.00	3.84±0.00
	ASAP	4.09±0.20	0.41±0.03	3.30±0.09	2.93±0.20	**2.83±0.12**	0.47±0.02	2.67±0.05	2.02±0.15	**2.49±0.08**	0.31±0.01	2.81±0.01	2.35±0.09
	RWM	4.71±0.15	0.40±0.03	3.15±0.07	2.91±0.11	4.71±0.28	0.32±0.03	2.55±0.07	2.98±0.38	4.37±0.64	0.36±0.14	2.11±0.18	2.33±0.19
	HumanoidNDM	**3.59±0.24**	**0.27±0.03**	**2.15±0.07**	**2.56±0.23**	2.90±0.13	**0.22±0.02**	**1.80±0.04**	**1.86±0.13**	**2.49±0.09**	**0.13±0.01**	**1.54±0.01**	**1.55±0.08**
Balance	PPO w/ Disturb.	2.66	0.56	3.61	1.90	2.46	0.39	3.39	2.01	2.52	0.33	3.04	1.78
	Direct Transfer	4.32±0.00	0.69±0.00	3.55±0.00	4.24±0.01	5.88±0.02	0.76±0.00	3.32±0.00	6.72±0.03	6.04±0.02	0.72±0.00	3.28±0.00	7.71±0.02
	ASAP	**3.40±0.14**	0.70±0.04	3.89±0.22	2.62±0.19	**2.61±0.22**	**0.33±0.02**	3.35±0.11	**3.18±0.55**	**2.66±0.21**	0.40±0.02	3.36±0.08	**1.90±0.13**
	RWM	4.00±0.10	0.52±0.03	**3.32±0.18**	3.62±0.29	5.03±0.43	0.42±0.02	3.24±0.08	3.77±0.29	4.77±0.41	0.36±0.04	3.07±0.09	3.25±0.34
	HumanoidNDM	3.51±0.13	**0.46±0.05**	3.45±0.16	**2.32±0.16**	3.82±0.27	0.49±0.02	**3.21±0.12**	4.07±0.49	3.24±0.15	**0.35±0.02**	**2.94±0.06**	2.77±0.12
Squat	PPO w/ Disturb.	5.59	1.04	2.25	5.50	2.46	0.17	2.05	5.22	1.74	0.17	1.94	4.67
	Direct Transfer	8.97±0.00	1.50±0.00	3.39±0.00	6.20±0.00	5.97±0.01	0.95±0.00	2.66±0.00	6.99±0.02	2.13±0.00	0.10±0.00	1.70±0.00	4.00±0.00
	ASAP	7.55±0.26	1.20±0.10	3.23±0.11	5.88±0.68	**3.86±0.23**	0.38±0.04	2.56±0.09	5.82±0.22	**1.86±0.27**	**0.05±0.02**	1.63±0.11	4.02±0.51
	RWM	7.20±0.20	0.97±0.11	2.91±0.15	5.38±0.57	4.75±0.56	0.54±0.16	2.52±0.23	5.98±0.51	2.56±0.00	0.11±0.00	1.94±0.19	4.51±0.58
	HumanoidNDM	**5.98±0.30**	**0.78±0.07**	**2.44±0.08**	**4.65±0.46**	4.44±0.21	**0.34±0.04**	**2.39±0.08**	**4.76±0.17**	2.09±0.24	0.08±0.02	**1.52±0.08**	**3.93±0.44**
Swing	PPO w/ Disturb.	2.06	0.21	1.50	1.48	1.96	0.28	2.28	2.90	1.07	0.10	0.87	0.83
	Direct Transfer	4.52±0.00	0.70±0.00	3.01±0.00	2.81±0.00	4.35±0.00	0.86±0.00	2.85±0.00	6.26±0.00	2.16±0.00	0.14±0.00	2.39±0.00	0.90±0.00
	ASAP	4.15±0.51	0.68±0.06	2.71±0.31	2.54±0.21	3.96±0.30	0.79±0.01	3.31±0.09	6.30±0.18	2.02±0.03	0.12±0.00	2.22±0.05	0.82±0.07
	RWM	3.79±0.24	0.40±0.08	2.01±0.04	1.85±0.22	3.33±0.16	0.51±0.05	2.14±0.07	4.79±0.23	1.28±0.03	**0.03±0.00**	1.19±0.05	**0.50±0.03**
	HumanoidNDM	**2.40±0.37**	**0.17±0.06**	**1.64±0.28**	**1.38±0.25**	**2.06±0.28**	**0.15±0.01**	**1.52±0.09**	**3.09±0.19**	**1.18±0.03**	**0.03±0.00**	**0.95±0.04**	0.63±0.06
Stride	PPO w/ Disturb.	3.35	0.50	2.83	5.75	4.06	0.63	3.76	7.12	3.92	0.62	2.89	6.24
	Direct Transfer	4.24±0.01	0.66±0.00	3.72±0.00	6.96±0.02	5.64±0.00	1.11±0.00	4.31±0.00	7.27±0.01	4.63±0.01	0.78±0.00	3.94±0.00	6.83±0.02
	ASAP	4.22±0.07	0.64±0.02	3.36±0.04	7.03±0.29	5.36±0.24	**0.72±0.02**	3.83±0.10	7.71±0.36	4.16±0.06	**0.44±0.02**	**3.19±0.04**	**6.02±0.06**
	RWM	5.10±0.01	0.53±0.16	**3.06±0.14**	**5.40±0.16**	6.07±0.24	0.77±0.11	**3.36±0.07**	**5.98±0.37**	4.58±0.03	0.63±0.03	3.62±0.08	6.76±0.20
	HumanoidNDM	**4.15±0.06**	**0.52±0.02**	3.42±0.04	6.25±0.24	**5.35±0.18**	0.96±0.03	4.05±0.07	6.88±0.26	**4.01±0.05**	0.72±0.02	3.73±0.04	6.73±0.04

that with only 200 deployment trajectories, HumanoidNDM can model the target dynamics with high fidelity, effectively bridging the training-deployment gap without extensive interactions.

5.1.2 Ablation Study

Comparison of Exploration Strategies We replace FENCE in the data collection phase of HumanoidNDM with other classic methods. Figure 3 shows normalized $E_{\text{link_pos}}$ performance curves during fine-tuning (left) and average collection smoothness (right). Although both FENCE and Action OU noise achieve the best fine-tuning results, FENCE significantly outperforms Action OU in S_{qpos} and $S_{\text{roll-pitch}}$, achieving stability second only to the no exploration baseline. This proves that FENCE effectively improves data collection stability while ensuring sufficient state-action coverage to support policy fine-tuning.

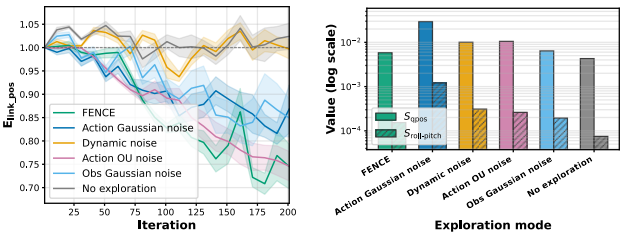


Figure 3: Comparison between exploration methods.

To demonstrate the generalizability of FENCE across different training frameworks, we apply it to ASAP and RWM baselines. The original ASAP implementation relies on random resets for data diversity, which is impractical in real world. For the default RWM, we use action Gaussian noise for data collection. As shown in Table 3, replacing these default strategies with FENCE maintains the fine-tuning performance. Furthermore, FENCE achieves data collection stability comparable to the impractical random reset approach, while significantly outperforming action Gaussian noise.

Table 3: Effectiveness of FENCE on baselines (mean±std).

Method	Policy Evaluation Performance		Data Collection Performance	
	$E_{\text{link_pos}}\downarrow$	$E_{\text{qpos}}\downarrow$	S_{qpos} $(\times 10^{-3})\downarrow$	$S_{\text{roll-pitch}}$ $(\times 10^{-4})\downarrow$
ASAP	3.68±0.20	**3.03±0.10**	**4.79±6.47**	**0.95±3.06**
ASAP w/ FENCE	**3.53±0.18**	3.13±0.08	5.84±6.27	1.23±4.25
RWM	4.42±0.23	**2.68±0.11**	27.44±13.82	11.06±9.73
RWM w/ FENCE	**4.23±0.35**	2.72±0.13	**5.43±5.12**	**1.48±4.19**

Dual Space Objective vs. Configuration-space-only Objective The two leftmost plots in Figure 4 compare optimizing with dual-space and configuration-space-only objectives. As shown, optimizing solely in the configuration space performs significantly worse than the dual-space approach, degrading both the Euclidean ($E_{\text{link_pos}}$) and configuration (E_{qpos}) metrics.

History-based World Model v.s. Single Frame World Model The right plots of Figure 4 compare the history-based neural dynamics model against a single-frame variant. We can see that re-

Table 4: Performance under added weight disturbances (top) and hard-to-model disturbances (bottom) (mean±std). The third column represents the task-specific metric reflecting task completeness.

Setting	Swallow Balance (w/ leg weights)			Arm Swing (w/ hand weights)			Pen Capping (w/ hand weight)		
Method	$E_{\text{link_pos}} \downarrow$	$E_{\text{qpos}} \downarrow$	air time (s) ↑	$E_{\text{link_pos}} \downarrow$	$E_{\text{qpos}} \downarrow$	arm height (m) ↑	$E_{\text{link_pos}} \downarrow$	$E_{\text{qpos}} \downarrow$	EE distance (cm) ↓
PPO w/ Disturb.	4.72 ± 0.28	$\mathbf{2.90 \pm 0.39}$	0.40 ± 0.11	$\mathbf{2.72 \pm 0.01}$	2.34 ± 0.03	$\mathbf{1.43 \pm 0.02}$	$\mathbf{6.58 \pm 0.00}$	$\mathbf{3.27 \pm 0.00}$	$\mathbf{0.00 \pm 0.00}$
Direct Transfer	5.34 ± 0.06	3.40 ± 0.07	0.00 ± 0.00	5.85 ± 0.02	3.33 ± 0.01	0.98 ± 0.00	6.96 ± 0.00	3.50 ± 0.00	3.50 ± 0.00
HumanoidNDM	$\mathbf{4.44 \pm 0.13}$	2.98 ± 0.34	$\mathbf{1.90 \pm 0.61}$	2.98 ± 0.00	$\mathbf{2.30 \pm 0.01}$	1.40 ± 0.01	6.82 ± 0.00	3.36 ± 0.00	$\mathbf{0.00 \pm 0.00}$

Setting	Lateral Raise (w/ elastic bands)			Walking (w/ water bucket)			Bowing (w/ damaged motor)		
Method	$E_{\text{link_pos}} \downarrow$	$E_{\text{qpos}} \downarrow$	arm height (m) ↑	$E_{\text{link_pos}} \downarrow$	$E_{\text{qpos}} \downarrow$	deviation angle (°) ↓	$E_{\text{link_pos}} \downarrow$	$E_{\text{qpos}} \downarrow$	noise level (dB) ↓
PPO w/ Disturb.	N/A			N/A			N/A		
Direct Transfer	7.54 ± 0.58	5.23 ± 0.19	1.06 ± 0.07	4.12 ± 0.22	3.67 ± 0.31	38 ± 17	4.40 ± 0.20	4.45 ± 0.14	73 ± 8
HumanoidNDM	$\mathbf{4.65 \pm 0.17}$	$\mathbf{4.40 \pm 0.03}$	$\mathbf{1.35 \pm 0.06}$	$\mathbf{4.01 \pm 0.27}$	$\mathbf{3.26 \pm 0.14}$	$\mathbf{5 \pm 3}$	$\mathbf{4.06 \pm 0.15}$	$\mathbf{4.01 \pm 0.03}$	$\mathbf{55 \pm 5}$

placing the traditional single-frame model with history-based architecture effectively enhances both training stability and overall tracking performance.

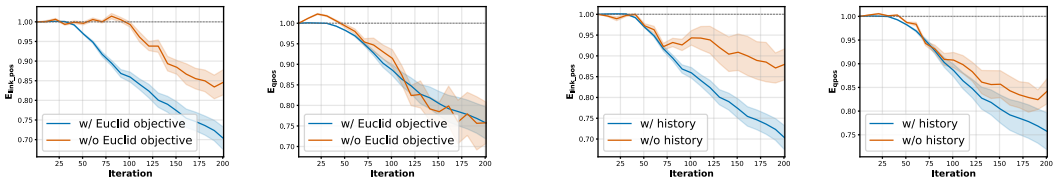


Figure 4: Left: Training w/ and w/o Euclidean-space objective. Right: Model w/ and w/o history.

5.2 Evaluation in Real-World

Performance Improvement Under Disturbances As shown in Figure 1, we select 6 motions with corresponding disturbances for real-world experiments. These encompass highly dynamic behaviors (walking, lateral raise, arm swing) and tasks emphasizing precise control (pen capping, swallow balance, bowing). We apply weight disturbances to 3 motions and hard-to-model disturbances to the rest. We use 50 real-world trajectories to fine-tune Pen Capping, and 200 for the others.

As illustrated in Table 4, HumanoidNDM achieves clear performance improvements across all 6 tasks. Each experiment is repeated 3 times. The gains are especially significant in our defined task-specific metrics, reflecting a substantial enhancement in motion completion. Notably, in the 3 settings with weight disturbances, our fine-tuned policy demonstrates performance comparable to, or even better than (e.g., Swallow Balance), the policy directly trained in simulation with explicitly modeled disturbances.

Comparison of Exploration Methods To demonstrate the superiority of FENCE in real-world data collection, we deploy different exploration methods for the lateral raise, where parameters remain the same to simulation. As shown in Table 5, the other 3 exploration methods cause obvious joint jitter and roll-pitch shaking, significantly widening the stability gap with FENCE. Meanwhile, FENCE remains nearly as stable as No Noise.

Table 5: Real-world comparison of exploration methods (mean±std).

Method	$S_{\text{qpos}} \downarrow$	$S_{\text{roll-pitch}} \downarrow$
FENCE	$\mathbf{0.099 \pm 0.018}$	$\mathbf{0.126 \pm 0.023}$
Action Gaussian Noise	1.275 ± 0.313	1.678 ± 0.455
Action OU Noise	0.707 ± 0.180	0.739 ± 0.210
Obs Gaussian Noise	1.151 ± 0.315	1.033 ± 0.218
No Noise	0.076 ± 0.005	0.106 ± 0.008

6 Conclusion and Limitations

We present HumanoidNDM, a framework that achieves the first online fine-tuning for humanoid tracking policies through deployment-viable exploration, Euclidean-space model training, and efficient model-gradient-guided policy fine-tuning. Experimental results show that HumanoidNDM effectively closes the training-deployment gap, enabling real-world adaptation to diverse disturbances. However, our current method is limited to time-invariant disturbances. In addition, future work could incorporate prior knowledge for more directed and efficient exploration.

Acknowledgments

This work was partially supported by New Generation Artificial Intelligence-National Science and Technology Major Project (No. 2025ZD0122905)

References

[1] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[2] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International conference on machine learning*, pages 1861–1870. Pmlr, 2018.

[3] P. M. Wensing, A. Wang, S. Seok, D. Otten, J. Lang, and S. Kim. Proprioceptive actuator design in the mit cheetah: Impact mitigation and high-bandwidth physical interaction for dynamic legged robots. *IEEE Transactions on Robotics*, 33(3):509–522, 2017. doi:10.1109/TRO.2016.2640183.

[4] B. Katz, J. Di Carlo, and S. Kim. Mini cheetah: A platform for pushing the limits of dynamic quadruped control. In *2019 International Conference on Robotics and Automation (ICRA)*, pages 6295–6301, 2019. doi:10.1109/ICRA.2019.8793865.

[5] M. Ji, X. Peng, F. Liu, J. Li, G. Yang, X. Cheng, and X. Wang. Exbody2: Advanced expressive humanoid whole-body control. *arXiv preprint arXiv:2412.13196*, 2024.

[6] T. He, Z. Luo, X. He, W. Xiao, C. Zhang, W. Zhang, K. Kitani, C. Liu, and G. Shi. Omnih2o: Universal and dexterous human-to-humanoid whole-body teleoperation and learning. *arXiv preprint arXiv:2406.08858*, 2024.

[7] Z. Zhang, J. Guo, C. Chen, J. Wang, C. Lin, Y. Lian, H. Xue, Z. Wang, M. Liu, J. Lyu, et al. Track any motions under any disturbances. *arXiv preprint arXiv:2509.13833*, 2025.

[8] Z. Chen, M. Ji, X. Cheng, X. Peng, X. B. Peng, and X. Wang. Gmt: General motion tracking for humanoid whole-body control. *arXiv preprint arXiv:2506.14770*, 2025.

[9] Q. Liao, T. E. Truong, X. Huang, Y. Gao, G. Tevet, K. Sreenath, and C. K. Liu. Beyondmimic: From motion tracking to versatile humanoid control via guided diffusion. *arXiv preprint arXiv:2508.08241*, 2025.

[10] A. Raffin, J. Kober, and F. Stulp. Smooth exploration for robotic reinforcement learning. In *Conference on robot learning*, pages 1634–1644. PMLR, 2022.

[11] J. Muru and A. Rassõlkin. A scoping review of energy consumption in industrial robotics. *Machines*, 13(7):542, 2025.

[12] T. He, J. Gao, W. Xiao, Y. Zhang, Z. Wang, J. Wang, Z. Luo, G. He, N. Sobanbab, C. Pan, et al. Asap: Aligning simulation and real-world physics for learning agile humanoid whole-body skills. *arXiv preprint arXiv:2502.01143*, 2025.

[13] L. Fussell, K. Bergamin, and D. Holden. Supertrack: Motion tracking for physically simulated characters using supervised learning. *ACM Transactions on Graphics (TOG)*, 40(6):1–13, 2021.

[14] H. Yao, Z. Song, B. Chen, and L. Liu. Controlvae: Model-based learning of generative controllers for physics-based characters. *ACM Transactions on Graphics (TOG)*, 41(6):1–16, 2022.

[15] D. Hafner, T. Lillicrap, J. Ba, and M. Norouzi. Dream to control: Learning behaviors by latent imagination. *arXiv preprint arXiv:1912.01603*, 2019.

[16] D. Hafner, T. Lillicrap, M. Norouzi, and J. Ba. Mastering atari with discrete world models. *arXiv preprint arXiv:2010.02193*, 2020.

[17] D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023.

[18] J. Xu, V. Makoviychuk, Y. Narang, F. Ramos, W. Matusik, A. Garg, and M. Macklin. Accelerated policy learning with parallel differentiable simulation. *arXiv preprint arXiv:2204.07137*, 2022.

[19] X. Wang, C. Zhang, W. Xie, C. Yu, W. Song, C. Bai, and S. Zhu. Halo: Closing sim-to-real gap for heavy-loaded humanoid agile motion skills via differentiable simulation. *arXiv preprint arXiv:2603.15084*, 2026.

[20] N. Sobanbabu, G. He, T. He, Y. Yang, and G. Shi. Sampling-based system identification with active exploration for legged robot sim2real learning. *arXiv preprint arXiv:2505.14266*, 2025.

[21] N. Fey, G. B. Margolis, M. Peticco, and P. Agrawal. Bridging the sim-to-real gap for athletic loco-manipulation. *arXiv preprint arXiv:2502.10894*, 2025.

[22] X. Liu, H. Wang, and L. Yi. Dexndm: Closing the reality gap for dexterous in-hand rotation via joint-wise neural dynamics model. *arXiv preprint arXiv:2510.08556*, 2025.

[23] L. Smith, I. Kostrikov, and S. Levine. A walk in the park: Learning to walk in 20 minutes with model-free reinforcement learning. *arXiv preprint arXiv:2208.07860*, 2022.

[24] L. Smith, J. C. Kew, X. B. Peng, S. Ha, J. Tan, and S. Levine. Legged robots that keep on learning: Fine-tuning locomotion policies in the real world. In *2022 international conference on robotics and automation (ICRA)*, pages 1593–1599. IEEE, 2022.

[25] L. Smith, Y. Cao, and S. Levine. Grow your limits: Continuous improvement with real-world rl for robotic locomotion. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 10829–10836. IEEE, 2024.

[26] T.-Y. Yang, T. Zhang, L. Luu, S. Ha, J. Tan, and W. Yu. Safe reinforcement learning for legged locomotion. In *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 2454–2461. IEEE, 2022.

[27] K. Hu, H. Shi, Y. He, W. Wang, C. K. Liu, and S. Song. Robot trains robot: Automatic real-world policy adaptation and learning for humanoids. *arXiv preprint arXiv:2508.12252*, 2025.

[28] M. Bloesch, J. Humplik, V. Patraucean, R. Hafner, T. Haarnoja, A. Byravan, N. Y. Siegel, S. Tunyasuvunakool, F. Casarini, N. Batchelor, et al. Towards real robot learning in the wild: A case study in bipedal locomotion. In *Conference on Robot Learning*, pages 1502–1511. PMLR, 2022.

[29] J. Levy, T. Westenbroek, K. Huang, F. Palafox, P. Yin, S. Omidshafiei, D.-K. Kim, A. Gupta, and D. Fridovich-Keil. Simulation distillation: Pretraining world models in simulation for rapid real-world adaptation. *arXiv preprint arXiv:2603.15759*, 2026.

[30] J. Yang, K. Lin, J. Li, W. Zhang, T. Lin, L. Wu, Z. Su, H. Zhao, Y.-Q. Zhang, L. Chen, et al. Rise: Self-improving robot policy with compositional world model. *arXiv preprint arXiv:2602.11075*, 2026.

[31] C. Li, A. Krause, and M. Hutter. Robotic world model: A neural network simulator for robust policy optimization in robotics. *arXiv preprint arXiv:2501.10100*, 2025.

[32] C. Li, A. Krause, and M. Hutter. Uncertainty-aware robotic world model makes offline model-based reinforcement learning work on real robots. *arXiv preprint arXiv:2504.16680*, 2025.

[33] W. Huang, Z. Li, H. Liu, B. Hou, Y. Su, and J. Zhang. Towards bridging the gap between large-scale pretraining and efficient finetuning for humanoid control. *arXiv preprint arXiv:2601.21363*, 2026.

[34] J. Levy, T. Westenbroek, and D. Fridovich-Keil. Learning to walk from three minutes of real-world data with semi-structured dynamics models. *arXiv preprint arXiv:2410.09163*, 2024.

[35] H. Shi, T. Li, Q. Zhu, J. Sheng, L. Han, and M. Q.-H. Meng. An efficient model-based approach on learning agile motor skills without reinforcement. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 5724–5730. IEEE, 2024.

[36] L. Wang, X. Zhang, Y. Wang, G. Zhan, W. Wang, H. Gao, J. Duan, and S. E. Li. Off-policy reinforcement learning with model-based exploration augmentation. *Advances in Neural Information Processing Systems*, 38:92121–92151, 2026.

[37] S. Amin, M. Gomrokchi, H. Satija, H. Van Hoof, and D. Precup. A survey of exploration methods in reinforcement learning. *arXiv preprint arXiv:2109.00157*, 2021.

[38] J. S. Achiam. *Exploration and safety in deep reinforcement learning*. University of California, Berkeley, 2021.

[39] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra. Continuous control with deep reinforcement learning. In Y. Bengio and Y. LeCun, editors, *4th International Conference on Learning Representations, ICLR 2016, San Juan, Puerto Rico, May 2-4, 2016, Conference Track Proceedings*, 2016. URL http://arxiv.org/abs/1509.02971.

[40] O. Eberhard, J. Hollenstein, C. Pinneri, and G. Martius. Pink noise is all you need: Colored noise exploration in deep reinforcement learning. In *The Eleventh International Conference on Learning Representations*, 2023.

[41] W. Songwei, J. Zhiduo, X. Guanghu, S. Wandong, L. Hong, and L. Yang. Cola-flow policy: Temporally coherent imitation learning via continuous latent action flow matching for robotic manipulation. *arXiv e-prints*, pages arXiv–2601, 2026.

[42] P. Ladosz, L. Weng, M. Kim, and H. Oh. Exploration in deep reinforcement learning: A survey. *Information Fusion*, 85:1–22, 2022.

[43] P. Yin, T. Westenbroek, S. Bagaria, K. Huang, C.-a. Cheng, A. Kobolov, and A. Gupta. Rapidly adapting policies to the real world via simulation-guided fine-tuning. *arXiv preprint arXiv:2502.02705*, 2025.

[44] L. Smith, Y. Cao, and S. Levine. Grow your limits: Continuous improvement with real-world rl for robotic locomotion. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 10829–10836. IEEE, 2024.

[45] A. G. Barto. Reinforcement learning: An introduction. by richard's sutton. *SIAM Rev*, 6(2): 423, 2021.

[46] R. J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine learning*, 8(3):229–256, 1992.

[47] R. Khraishi and R. Okhrati. Simple noisy environment augmentation for reinforcement learning. *arXiv preprint arXiv:2305.02882*, 2023.

[48] Y.-T. Li, J. Lin, J. Cheng, and P. Pachuca. Neighboring state-based exploration for reinforcement learning, 2025.

[49] K. Zakka, B. Tabanpour, Q. Liao, M. Haiderbhai, S. Holt, J. Y. Luo, A. Allshire, E. Frey, K. Sreenath, L. A. Kahrs, et al. Mujoco playground. *arXiv preprint arXiv:2502.08844*, 2025.

[50] M. Janner, J. Fu, M. Zhang, and S. Levine. When to trust your model: Model-based policy optimization. *Advances in neural information processing systems*, 32, 2019.

[51] R. Wei, N. Lambert, A. McDonald, A. Garcia, and R. Calandra. A unified view on solving objective mismatch in model-based reinforcement learning. *arXiv preprint arXiv:2310.06253*, 2023.

[52] P. Ball, J. Parker-Holder, A. Pacchiano, K. Choromanski, and S. Roberts. Ready policy one: World building through active learning. In *International Conference on Machine Learning*, pages 591–601. PMLR, 2020.

[53] T. Yu, G. Thomas, L. Yu, S. Ermon, J. Y. Zou, S. Levine, C. Finn, and T. Ma. Mopo: Model-based offline policy optimization. *Advances in neural information processing systems*, 33: 14129–14142, 2020.

[54] A. Ecoffet, J. Huizinga, J. Lehman, K. O. Stanley, and J. Clune. First return, then explore. *Nature*, 590(7847):580–586, 2021.

[55] J. Xu, E. Heiden, I. Akinola, D. Fox, M. Macklin, and Y. Narang. Neural robot dynamics. *arXiv preprint arXiv:2508.15755*, 2025.

[56] C. Jing, J. K. Bandi, J. Ye, Y. Duan, P. Abbeel, X. Wang, and S. Yi. Contact-aware neural dynamics. *arXiv preprint arXiv:2601.12796*, 2026.

[57] X. B. Peng, P. Abbeel, S. Levine, and M. Van de Panne. Deepmimic: Example-guided deep reinforcement learning of physics-based character skills. *ACM Transactions On Graphics (TOG)*, 37(4):1–14, 2018.

A Implementation Details of HumanoidNDM

A.1 Deployment-viable Real-world Exploration with Frequency-Domain Perturbations

A.1.1 Weighted Policy Neighborhood Coverage Metric

As introduced in the main text, we formulate the online adjustment of perturbation intensity σ as a Multi-Armed Bandit (MAB) problem over K discrete levels $\Sigma = \{\sigma_1, \ldots, \sigma_K\}$.

However, coverage across all timesteps should not be treated equally during an episode. For instance, aggressive exploration is unnecessary during a steady standing phase where tracking errors are minimal. To address this, we compute a weighted Policy Neighborhood Coverage metric $\bar{C}$, using the step-wise global positional tracking error e_t as the weighting factor:

$$\bar{C} = \frac{\sum_{t=1}^{T} e_t C_t}{\sum_{t=1}^{T} e_t} \tag{7}$$

where

$$C_t = \frac{1}{N_r N_d} \sum_{j=1}^{N_r} \sum_{k=1}^{N_d} \mathbb{I}\left(\mathcal{B} \cap B_t^{(j,k)} \neq \emptyset\right) \tag{8}$$

In our implementation, the number of radial bins N_r and directional bins N_d are maintained as constant values across all trajectories. Conversely, the neighborhood radius R is treated as a motion-specific hyperparameter. The rationale behind this design lies in the varying dependence of policy fine-tuning on the dynamics model's gradients. For motions characterized by relatively simple dynamics or stable contact states (e.g., *Swing*), the learned neural dynamics model typically exhibits robust interpolation and extrapolation capabilities. In such scenarios, the model can provide reliable, well-behaved gradients even if the collected data distribution is relatively sparse and widely dispersed, allowing us to adopt a larger radius R. On the other hand, for motions that necessitate fine-grained control and involve highly non-linear, sensitive dynamics (e.g., *Balance*), the model's capacity for accurate interpolation and extrapolation naturally diminishes. Consequently, a smaller R is required to enforce dense data coverage within a tightly localized neighborhood, ensuring the model accurately captures the precise dynamics necessary for effective policy fine-tuning.

For each candidate intensity σ_i, we maintain a localized recent buffer $\mathcal{B}_i$ to evaluate its local exploration quality. At each iteration, the empirical reward for choosing σ_i is evaluated by this weighted Policy Neighborhood Coverage score: $S(\sigma_i) = \bar{C}$.

A.1.2 Adaptive Intensity Adjustment via Multi-Armed Bandit

To effectively balance exploration of unknown intensity effects and exploitation of known optimal levels, we employ the Upper Confidence Bound (UCB) algorithm. The priority score $U(\sigma_i)$ for each arm is updated as follows:

$$U(\sigma_i) = S(\sigma_i) + \beta \sqrt{\frac{\ln N}{n_i}} \tag{9}$$

where n_i is the cumulative selection count for level σ_i, N is the total number of training iterations across all levels, and β is a hyperparameter controlling the exploration scale.

To achieve progressive exploration, the bandit is restricted to transition only between the current intensity level and its immediately adjacent levels. The optimal intensity for the subsequent iteration is then selected via:

$$\sigma^* = \arg \max_{\sigma_i \in \text{adjacent}} U(\sigma_i) \tag{10}$$

By iteratively tracking this argmax within adjacent bounds, FENCE dynamically adjusts the exploration intensity to maintain optimal data coverage without triggering mechanical instability.

A.2 MoCap-Free Neural Dynamics Model Training with Dual-Space Objective

A.2.1 Kinematically Constrained Configuration and Gravity Prediction

In our implementation, the world model w does not predict the next joint configuration or the gravity vector as fully independent, unconstrained outputs. To enforce physical and kinematic consistency during autoregressive rollouts, specific state components undergo tailored integration and coordinate transformation steps.

Joint Configuration Update Although the network outputs a residual vector with the same dimensionality as the processed state, the residual component corresponding to the joint position q_t is not directly applied. Instead, the model predicts the next velocity-like term via residual addition:

$$\hat{\dot{q}}_{t+1} = \dot{q}_t + \Delta\dot{q}_t$$

where $\Delta\dot{q}_t$ is the predicted joint velocity residual. The next joint position is subsequently obtained through first-order integration:

$$\hat{q}_{t+1} = q_t + \hat{\dot{q}}_{t+1}\Delta t$$

This design enforces a strict kinematic relationship between joint positions and velocities, preventing the model from producing arbitrary, discontinuous joint predictions across long rollout horizons.

Gravity Vector Roll-Pitch Transformation The onboard proprioceptive gravity vector $\mathbf{g}_t \in \mathbb{R}^3$ possesses geometric redundancy as it resides on a unit sphere, capturing only two underlying degrees of freedom. To eliminate this redundancy and facilitate smoother optimization, we map $\mathbf{g}_t$ into a 2D roll-pitch representation $\mathbf{r}_t = [\phi_t, \theta_t]^\top \in \mathbb{R}^2$ prior to feeding the history sequence into the history encoder:

$$\mathbf{r}_t = \mathcal{T}_{\text{3D}\to\text{2D}}(\mathbf{g}_t)$$

where $\mathcal{T}_{\text{3D}\to\text{2D}}$ denotes the transformation from the 3D gravity vector to Euler angles (roll and pitch). The neural dynamics model then operates on this low-dimensional space and outputs a 2D orientation residual $\Delta\mathbf{r}_t$. The predicted roll-pitch for the next time step is updated via residual addition:

$$\hat{\mathbf{r}}_{t+1} = \mathbf{r}_t + \Delta\mathbf{r}_t$$

Finally, the effective 3D gravity vector prediction $\hat{\mathbf{g}}_{t+1}$ is reconstructed by mapping $\hat{\mathbf{r}}_{t+1}$ back into the 3D Cartesian space:

$$\hat{\mathbf{g}}_{t+1} = \mathcal{T}_{\text{2D}\to\text{3D}}(\hat{\mathbf{r}}_{t+1})$$

Other unconstrained state components, such as the gyroscope signals $\boldsymbol{\omega}_t$ and the estimated root height h_t, are updated directly through standard residual addition.

A.2.2 Shared-Backbone Multi-Head Ensemble

To enhance predictive robustness while maintaining computational efficiency, the ensemble framework utilized in our method is implemented as a shared-backbone multi-head ensemble, rather than a collection of fully independent models.

All M ensemble members share the same history encoder and decoder backbone. The historical state-action sequence is encoded by a shared Transformer trunk. Ensemble diversity is injected solely at the final prediction heads, where each head maps the shared feature to a separate next-state prediction. During training, bootstrap sampling masks are applied independently at the head level, meaning different prediction heads are optimized on different sampled subsets of the same batch. Therefore, the ensemble should be interpreted as a bootstrapped multi-head dynamics model that captures prediction diversity through its output heads, significantly reducing the overhead compared to training M separate encoders and decoders.

A.2.3 Training Objectives of Neural Dynamics Model

The neural dynamics model w is trained with the following losses:

- $\mathcal{L}_{\omega}^{w}$: the root angular-velocity prediction loss,

$$\mathcal{L}_{\omega}^{w} = \|\hat{\boldsymbol{\omega}}_{t+k} - \boldsymbol{\omega}_{t+k}\|_2 .$$

  Here, $\hat{\boldsymbol{\omega}}_{t+k}$ and $\boldsymbol{\omega}_{t+k}$ denote the predicted and target root angular velocities, respectively.

- $\mathcal{L}_{\mathrm{rp}}^{w}$: the root roll–pitch prediction loss,

$$\mathcal{L}_{\mathrm{rp}}^{w} = \left\|\hat{\boldsymbol{\rho}}_{t+k} - \boldsymbol{\rho}_{t+k}\right\|_2 .$$

  Here, $\hat{\boldsymbol{\rho}}_{t+k}$ and $\boldsymbol{\rho}_{t+k}$ denote the predicted and target root roll–pitch angles, respectively.

- $\mathcal{L}_{q}^{w}$: the joint-position prediction loss,

$$\mathcal{L}_{q}^{w} = \|\hat{q}_{t+k} - q_{t+k}\|_2 .$$

  Here, $\hat{q}_{t+k}$ and q_{t+k} denote the predicted and target joint positions, respectively.

- $\mathcal{L}_{h}^{w}$: the root-height prediction loss,

$$\mathcal{L}_{h}^{w} = \left|\hat{h}_{t+k} - h_{t+k}\right| .$$

  Here, $\hat{h}_{t+k}$ and h_{t+k} denote the predicted and target root heights, respectively.

- $\mathcal{L}_{\mathrm{link_pos}}^{w}$: the link-position loss in a local gravity-aligned frame,

$$\mathcal{L}_{\mathrm{link_pos}}^{w} = \left\|\hat{\mathbf{P}}_{t+k} - \mathbf{P}_{t+k}\right\|_F .$$

  Here, $\hat{\mathbf{P}}_{t+k}, \mathbf{P}_{t+k} \in \mathbb{R}^{|\mathcal{B}|\times 3}$ denote the predicted and target local gravity-aligned frame positions of all robot links, respectively.

- $\mathcal{L}_{\mathrm{link_rot}}^{w}$: the link-rotation loss in a local gravity-aligned frame,

$$\mathcal{L}_{\mathrm{link_rot}}^{w} = \frac{1}{|\mathcal{B}|}\sum_{\ell\in\mathcal{B}} d_{\mathrm{geo}}^{2}\left(\hat{\mathbf{R}}_{\ell,t+k}, \mathbf{R}_{\ell,t+k}\right),$$

  where $\mathcal{B}$ denotes the set of robot links, and $\hat{\mathbf{R}}_{\ell,t+k}, \mathbf{R}_{\ell,t+k} \in SO(3)$ are the predicted and target local gravity-aligned frame rotations of link ℓ, respectively. The geodesic rotation distance is defined as

$$d_{\mathrm{geo}}(\mathbf{R}_1, \mathbf{R}_2) = \arccos\left(\frac{\mathrm{tr}(\mathbf{R}_1\mathbf{R}_2^{\top}) - 1}{2}\right).$$

A.3 Efficient and Stable Policy Fine-Tuning via Supervised Gradients

A.3.1 Initialization of Autoregressive Rollout

During policy fine-tuning, autoregressive rollouts are not initialized from the beginning of the reference trajectory or from a nominal default pose. Instead, each training rollout is initialized using a short history segment sampled directly from the buffer.

Concretely, the exploration buffer stores tuples comprising collected real-world states, executed actions, corresponding reference trajectory indices, and previous actions. For each policy update iteration, we sample a sequence consisting of a history prefix of length H and a rollout horizon of length L_π. The first H collected states $\mathbf{s}_{t-H+1:t}$ and $H-1$ collected actions $\mathbf{a}_{t-H+1:t-1}$ are extracted to initialize the history context for the dynamics model.

This buffer-initialization strategy ensures that policy fine-tuning optimizes the policy starting from states that were actually visited during real-world data collection. Consequently, it anchors the supervised-gradient updates locally to the target-domain state distribution covered by the exploration buffer. Furthermore, it supplies the history-conditioned world model with a realistic and dynamically consistent recent state-action context, which is crucial for accurate forward prediction.

A.3.2 Training Objectives of Policy

The policy π is fine-tuned with the following losses:

- $\mathcal{L}^{\pi}_{\mathrm{rp}}$: the root roll–pitch tracking loss,

$$\mathcal{L}^{\pi}_{\mathrm{rp}} = \left\| \hat{\boldsymbol{\rho}}^{\pi}_{t+k} - \boldsymbol{\rho}_{t+k} \right\|_2^2 .$$

  Here, $\hat{\boldsymbol{\rho}}^{\pi}_{t+k}$ and $\boldsymbol{\rho}_{t+k}$ denote the rollout and target root roll–pitch angles, respectively.

- $\mathcal{L}^{\pi}_{q}$: the joint-position tracking loss,

$$\mathcal{L}^{\pi}_{q} = \left\| \hat{q}^{\pi}_{t+k} - q_{t+k} \right\|_2^2 .$$

  Here, $\hat{q}^{\pi}_{t+k}$ and q_{t+k} denote the rollout and target joint positions, respectively.

- $\mathcal{L}^{\pi}_{\mathrm{sm}}$: the joint-space smoothness regularization loss,

$$\mathcal{L}^{\pi}_{\mathrm{sm}} = \left\| \hat{q}^{\pi}_{t+k} - 2\hat{q}^{\pi}_{t+k-1} + \hat{q}^{\pi}_{t+k-2} \right\|_2^2 .$$

  Here, $\hat{q}^{\pi}_{t+k}$, $\hat{q}^{\pi}_{t+k-1}$, and $\hat{q}^{\pi}_{t+k-2}$ denote the joint positions at three consecutive rollout steps.

- $\mathcal{L}^{\pi}_{\mathrm{link_pos}}$: the link-position loss in a local gravity-aligned frame,

$$\mathcal{L}^{\pi}_{\mathrm{link_pos}} = \left\| \hat{\mathbf{P}}^{\pi}_{t+k} - \mathbf{P}_{t+k} \right\|_F .$$

  Here, $\hat{\mathbf{P}}^{\pi}_{t+k}, \mathbf{P}_{t+k} \in \mathbb{R}^{|\mathcal{B}|\times 3}$ denote the rollout and target link positions of all robot links, respectively.

- $\mathcal{L}^{\pi}_{\mathrm{link_rot}}$: the link-rotation loss in a local gravity-aligned frame,

$$\mathcal{L}^{\pi}_{\mathrm{link_rot}} = \frac{1}{|\mathcal{B}|} \sum_{\ell \in \mathcal{B}} d^2_{\mathrm{geo}} \left(\hat{\mathbf{R}}^{\pi}_{\ell,t+k}, \mathbf{R}_{\ell,t+k} \right),$$

  where $\mathcal{B}$ denotes the set of robot links, and $\hat{\mathbf{R}}^{\pi}_{\ell,t+k}, \mathbf{R}_{\ell,t+k} \in SO(3)$ are the rollout and target rotations of link ℓ, respectively.

A.4 Hyperparameters of HumanoidNDM

Important hyperparameters of HumanoidNDM are listed in Table 6.

A.5 Architecture of the Neural Dynamics Model

The neural dynamics model employs a history-based Transformer ensemble architecture to predict future states. Let $s_t = [\boldsymbol{\omega}_t, \boldsymbol{\rho}_t, q_t, \dot{q}_t, h_t] \in \mathbb{R}^{64}$ denote the system state and $a_t \in \mathbb{R}^{23}$ denote the action. We utilize a history length of $H = 10$ and an ensemble size of $M = 5$. The architecture consists of a history encoder $\mathcal{E}_{\mathrm{hist}}$, a shared decoder $\mathcal{D}$, and independent prediction heads.

History Encoding To capture temporal dependencies, we process the past $H - 1$ state-action pairs. We first obtain the dimension-wise scaled state $\tilde{s}_t$ and construct input tokens x_i via a linear projection $\mathcal{P} : \mathbb{R}^{87} \to \mathbb{R}^{64}$:

$$x_i = \mathcal{P}\big([\tilde{s}_{t-H+i}, a_{t-H+i}]\big) + p_i, \qquad i = 1, \ldots, H-1,$$

where $p_i \in \mathbb{R}^{64}$ is a learnable positional embedding. These tokens are then processed by a Transformer encoder $\mathcal{E}_{\mathrm{hist}}$, which consists of $N_{\mathrm{enc}} = 2$ layers, $n_{\mathrm{head}} = 4$ attention heads, and a feedforward network width of $d_{\mathrm{ff}} = 16$. We extract the history feature $z_t \in \mathbb{R}^{64}$ from the final token:

$$z_t = \mathcal{E}_{\mathrm{hist}}(x_{1:H-1})_{H-1}.$$

Table 6: Hyperparameters of HumanoidNDM

Variable	Value
Data Collection	
Max perturbation frequency (k_c)	1.0 Hz
Roll-pitch termination threshold	1.0
Radial bin number	2
Directional bin number	16
MAB exploration scale (β)	0.05
Model Training	
History length (H)	10
Autoregressive rollout length (L_w)	8
Ensemble size (M)	5
Mini-batch size	512
Mini-batches per epoch	32
Model learning rate (η_w)	2×10^{-3}
Bootstrap probability	0.8
Replay-buffer horizon in episodes	100
Weight of $\mathcal{L}_{\omega}^{w}$	1.0
Weight of $\mathcal{L}_{\mathrm{rp}}^{w}$	1.0
Weight of $\mathcal{L}_{q}^{w}$	1.0
Weight of $\mathcal{L}_{h}^{w}$	1.0
Weight of $\mathcal{L}_{\mathrm{link_pos}}^{w}$	1.0
Weight of $\mathcal{L}_{\mathrm{link_rot}}^{w}$	5.0
Policy Training	
Policy learning rate (η_π)	10^{-4}
Learning-rate decay factor	1.0
Final learning rate ($\eta_\pi^{\mathrm{final}}$)	10^{-7}
Autoregressive rollout length (L_π)	128
Mini-batch size (B_π)	512
Mini-batches per epoch (N_{mb}^{π})	2
Weight of $\mathcal{L}_{q}^{\pi}$	10.0
Weight of $\mathcal{L}_{\mathrm{rp}}^{\pi}$	10.0
Weight of $\mathcal{L}_{\mathrm{sm}}^{\pi}$	10.0
Weight of $\mathcal{L}_{\mathrm{link_pos}}^{\pi}$	1.0
Weight of $\mathcal{L}_{\mathrm{link_rot}}^{\pi}$	1.0

Decoder and Ensemble Prediction The history feature z_t is concatenated with the current scaled state $\tilde{s}_t$ and action a_t, forming a 151-dimensional input vector. This vector is passed through a shared multilayer perceptron (MLP) decoder $\mathcal{D}$:

$$u_t = \mathcal{D}([z_t, \tilde{s}_t, a_t]), \qquad \mathcal{D} : 151 \to 64 \to 512 \to 512,$$

where the Exponential Linear Unit (ELU) activation function is applied between the hidden layers.

To model dynamics uncertainty, each member of the ensemble utilizes an independent linear prediction head $g_m : \mathbb{R}^{512} \to \mathbb{R}^{64}$ to predict the residual of the next state:

$$\Delta\tilde{s}_{t+1}^{(m)} = g_m(u_t), \qquad m = 1, \ldots, M.$$

State Integration The next state for each ensemble member is derived through residual prediction for most state components, while the joint positions are explicitly integrated from the predicted joint velocities using a time step of $\Delta t = 0.02\,\mathrm{s}$:

$$\begin{aligned}
\hat{\boldsymbol{\omega}}_{t+1}^{(m)} &= \boldsymbol{\omega}_t + \Delta\boldsymbol{\omega}_{t+1}^{(m)}, \\
\hat{\boldsymbol{\rho}}_{t+1}^{(m)} &= \boldsymbol{\rho}_t + \Delta\boldsymbol{\rho}_{t+1}^{(m)}, \\
\hat{\dot{q}}_{t+1}^{(m)} &= \dot{q}_t + \Delta\dot{q}_{t+1}^{(m)}, \\
\hat{h}_{t+1}^{(m)} &= h_t + \Delta h_{t+1}^{(m)}, \\
\hat{q}_{t+1}^{(m)} &= q_t + \hat{\dot{q}}_{t+1}^{(m)}\Delta t.
\end{aligned}$$

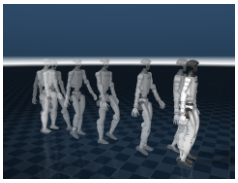
B Implementation of Baselines

ASAP We ported the official implementation of ASAP to the MuJoCo-Jax simulator. During this migration, we strictly preserved all original Proximal PPO training hyperparameters and reward formulations. Empirically, we observed that the fine-tuning performance is sensitive to the penalty coefficients for the action rate and action norm. Consequently, we conducted a systematic grid search over these two reward parameters for each experimental setting. All reported results for ASAP are averaged over 5 independent trials, utilizing the optimally identified hyperparameter configuration.

RWM Similarly, we integrated the official implementation of RWM into the MuJoCo-Jax simulator. While we retained the default PPO training configurations from the original codebase, the loss functions were aligned to be strictly consistent with those employed in HumanoidNDM. Our preliminary evaluations indicated that the rollout length, policy learning rate, and the initial standard deviation of the action distribution critically influence the downstream fine-tuning efficacy. Therefore, a grid search was performed across these three hyperparameters under every setting. All reported results for RWM are averaged over 5 independent trials, utilizing the optimally identified hyperparameter configuration.

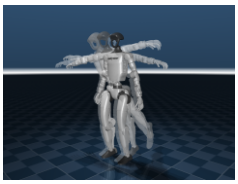
C Simulation Experiment Settings

We designed 5 motions paired with 3 kinds of disturbances in simulation experiments.



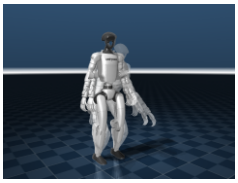
Walk Starting from the default pose, the robot takes four steps directly forward, stops, and returns to the initial default pose. Natural arm swings are maintained throughout motion.

- *Extra Weight:* The mass of the `torso` is multiplied by 1.2, and its local center of mass is offset by [-0.2, 0, 0].
- *Scaled KP:* The KP of the `waist_yaw` is decreased from 300 to 0.
- *Small Friction:* The coefficient of friction between the robot's feet and the ground is reduced from 1.0 to 0.3.



Balance Starting from the default pose, the robot lifts its left leg while leaning its torso forward to maintain equilibrium. This posture is held for approximately 2 seconds before the left leg is slowly lowered, returning the robot to the default pose.

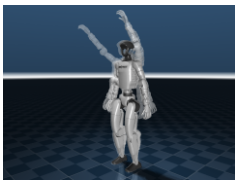
- *Extra Weight:* The mass of the `right_knee` is multiplied by 1.5, and its local center of mass is offset by [0, -0.2, 0].
- *Scaled KP:* The KP of the `left_hip_pitch` is increased from 100 to 300.
- *Small Friction:* The coefficient of friction between the robot's feet and the ground is reduced from 1.0 to 0.3.



Squat Starting from the default pose, the robot raises both arms forward and simultaneously executes a squatting motion. After approximately 2 seconds, the robot stands upright, lowers its arms, and returns to the default pose.

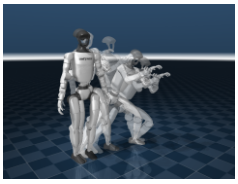
- *Extra Weight:* The masses of the `left_shoulder_roll`, `left_shoulder_yaw`, `left_elbow`, `left_wrist_roll` are multiplied by 8, and their respective local centers of mass are offset by [0, -0.3, 0].
- *Scaled KP:* The KP of the `right_hip_pitch` is decreased from 100 to 30.
- *Small Friction:* The coefficient of friction between the robot's feet and the ground is reduced from 1.0 to 0.3.

Swing Starting from the default pose, the robot raises its right arm vertically while swinging its left arm slightly backward. It then performs two rhythmic, small-amplitude extensions in this configuration, mimicking a dynamic chest expansion exercise. Following this, the robot executes the mirrored motion by swapping the positions of its arms, before finally returning to the default pose.



- *Extra Weight:* The masses of the `left_shoulder_roll`, `left_shoulder_yaw`, `left_elbow`, `left_wrist_roll` are multiplied by 4, and their respective local centers of mass are offset by [0, -0.2, 0].
- *Scaled KP:* The KP of the `left_shoulder_pitch` is decreased from 90 to 10.
- *Small Friction:* The coefficient of friction between the robot's feet and the ground is reduced from 1.0 to 0.3.

Stride Starting from the default pose, the robot brings both arms close to its chest and steps its right foot diagonally into a forward-right lunge. It then retracts the right leg and returns to the default pose.



- *Extra Weight:* The mass of the `torso` is multiplied by 1.2, and its local center of mass is offset by [-0.2, 0, 0].
- *Scaled KP:* The KP of the `left_hip_roll` is decreased from 100 to 30.
- *Small Friction:* The coefficient of friction between the robot's feet and the ground is reduced from 1.0 to 0.3.

D Real-World Experiment Settings

To evaluate HumanoidNDM in the real world, we designed six real-world tasks. We divided them into two groups based on the type of disturbance: **Added Weight Disturbances** (which are easier to simulate) and **Hard-to-Model Disturbances** (which involve complex dynamics like elasticity, fluid sloshing, and damaged motors). The detailed settings are described below.

NDM and policy training are performed on a single RTX 4090 GPU. Each adaptation iteration takes approximately 3 s for NDM and policy optimization and 10 s for data collection. The latter includes returning the robot to its initial position. Thus, 200 iterations can be completed within one hour. All reported real-world results are averaged over 3 independent trials, with no failed runs. Occasional interruptions caused by environmental collisions or motor overheating were handled by restoring the latest checkpoint and resuming training.

D.1 Added Weight Disturbances

In this category, we attached weights to specific links of the robot.

Swallow Balance

- *Action Description:* The robot holds its arms out and lifts its left leg back.
- *Disturbance:* A 1.5 kg iron weight is strapped to each calf.
- *Collected Real-World Data:* 200 trajectories.
- *Task-specific Metric (Air Time ↑):* Measured in seconds (s). This shows how long the left foot stays in the air. A longer time means better balance.

Arm Swing

- *Action Description:* The robot swings its right arm up, then its left arm, like marching.

- *Disturbance:* A 1.5 kg iron weight is attached to the left hand.
- *Collected Real-World Data:* 200 trajectories.
- *Task-specific Metric (Arm Height ↑):* Measured in meters (m). This is the highest point the loaded hand reaches. It shows if the loaded arms can lift the extra weight.

Pen Capping

- *Action Description:* The robot holds a pen cap in its left hand and a pen in its right. It brings them together to cap the pen, then moves its arms back down.
- *Disturbance:* A 0.75 kg weight is placed only on the left hand.
- *Collected Real-World Data:* 50 trajectories.
- *Task-specific Metric (EE Distance ↓):* Measured in centimeters (cm). This is the shortest distance between the pen and cap. A smaller distance means better coordination.

D.2 Hard-to-Model Disturbances

In this category, we apply disturbances that are hard to model in simulation, including external elastic forces, unpredictable weight shifts from sloshing fluids, and hardware defects such as damaged motors. These experiments further prove the necessity of real-world fine-tuning.

Lateral Raise

- *Action Description:* The robot raises both arms to the side to head height, then lowers them. It does this twice.
- *Disturbance:* Two elastic bands connect the right hand to the left and right wrists. This adds elastic forces.
- *Collected Real-World Data:* 200 trajectories.
- *Task-specific Metric (Arm Height ↑):* Measured in meters (m). This is the highest point the right arm reaches. It tests if the robot can resist the pulling forces.

Walking

- *Action Description:* The robot walks forward four steps and swings its arms naturally.
- *Disturbance:* The robot holds a bucket with 2 kg of water in its right hand. The sloshing water shifts the weight unpredictably.
- *Collected Real-World Data:* 200 trajectories.
- *Task-specific Metric (Deviation Angle ↓):* Measured in degrees (°). This shows how much the robot turns away from a straight path. A smaller angle means it handles the sloshing water better.

Bowing

- *Action Description:* The robot steps back with its right foot, draws two circles with its right hand, brings the hand to the front, and bows.
- *Disturbance:* We use a robot with damaged motors, which causes obvious stuttering.
- *Collected Real-World Data:* 200 trajectories.
- *Task-specific Metric (Noise Level ↓):* Measured in decibels (dB). A lower noise level means the policy generates smoother actions.

E Simulation Training Curves

The training curves of HumanoidNDM and baselines are shown in Figures 5– 9.

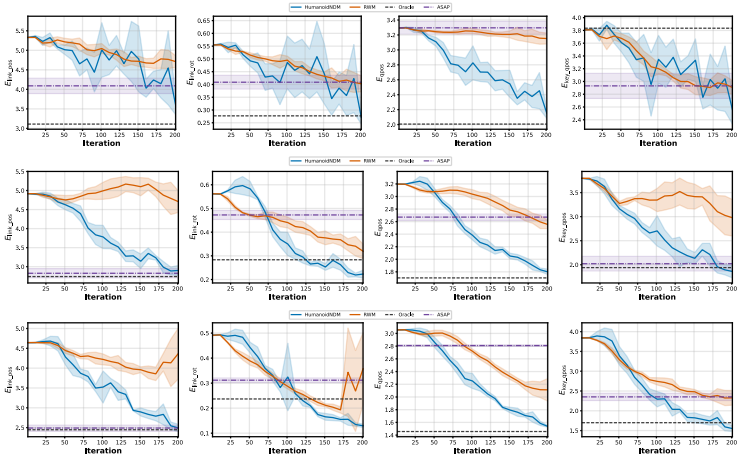


Figure 5: Walk: Extra Weight (top), Scaled KP (middle), Small Friction (bottom).

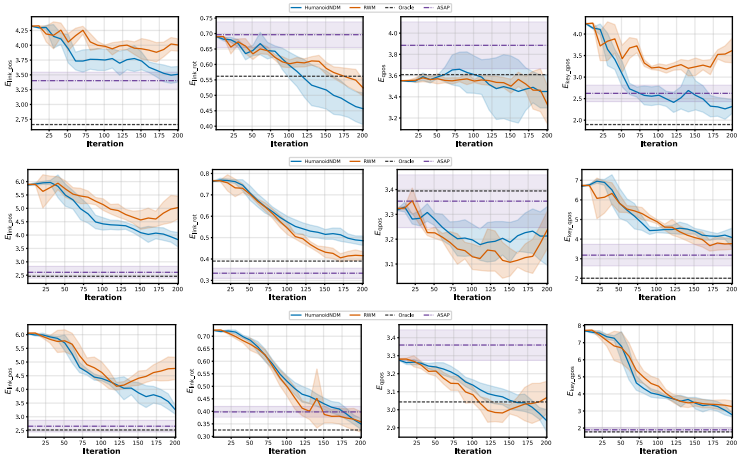


Figure 6: Balance: Extra Weight (top), Scaled KP (middle), Small Friction (bottom).

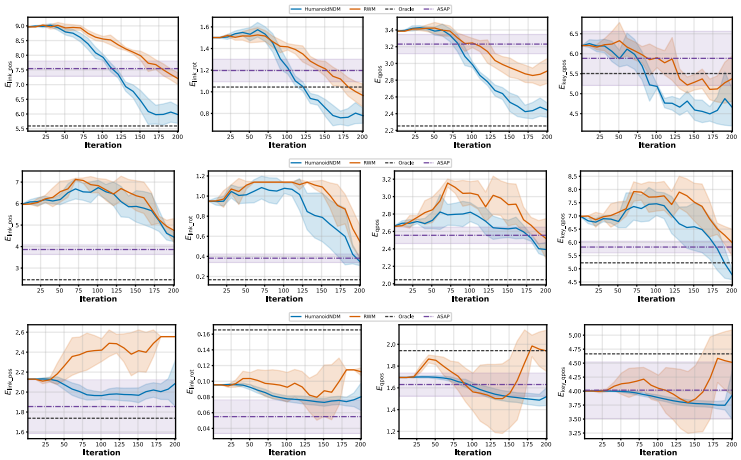


Figure 7: Squat: Extra Weight (top), Scaled KP (middle), Small Friction (bottom).

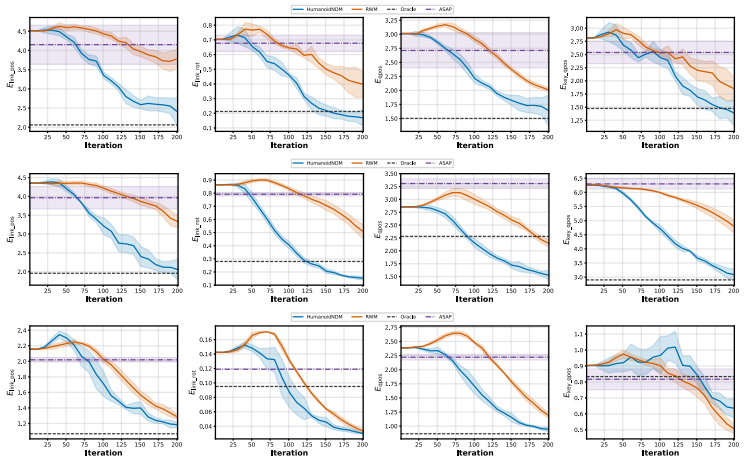


Figure 8: Swing: Extra Weight (top), Scaled KP (middle), Small Friction (bottom).

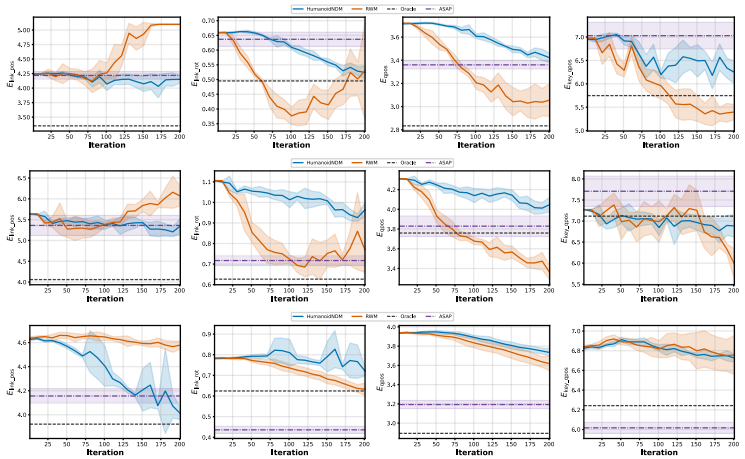


Figure 9: Stride: Extra Weight (top), Scaled KP (middle), Small Friction (bottom).

F Analysis of Perturbation Intensity Adaptation

In the simulation setting of our main experiments, we compare adaptive perturbation intensity with fixed scales, as shown in Figure 10. Fixed σ values of 2, 3, and 4 achieve similar fine-tuning performance, significantly outperforming $\sigma = 1$ (insufficient exploration) and $\sigma = 6$ (overly aggressive exploration). Adaptive intensity achieves nearly identical performance to these well-chosen fixed scales. We attribute this similarity to the initial policies' reward design, which produces nearly indistinguishable policy responses to reference perturbations within a reasonable range of intensities.

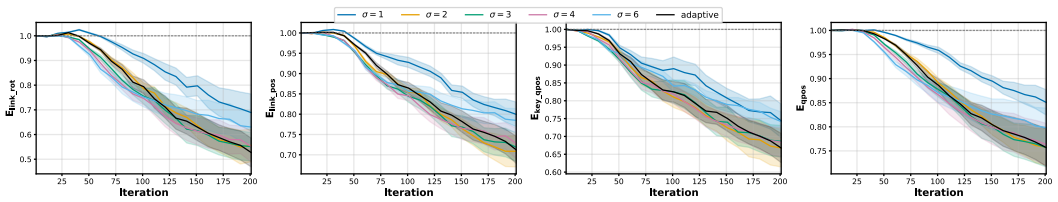


Figure 10: Comparison between the adaptive and fixed σ under the default simulation setting.

However, adaptive intensity demonstrates clearer advantages in real-world experiments. As shown in Figure 11, adaptive exploration outperforms all fixed perturbation scales for both stride and swing

motions. We hypothesize that this advantage arises from matching perturbation intensity to exploration requirements at different stages. Larger perturbations facilitate broader exploration in early stages, whereas smaller perturbations become sufficient as training approaches convergence, particularly for inherently less stable motions such as stride. Excessive perturbations near convergence may instead introduce unstable data and hinder fine-tuning.

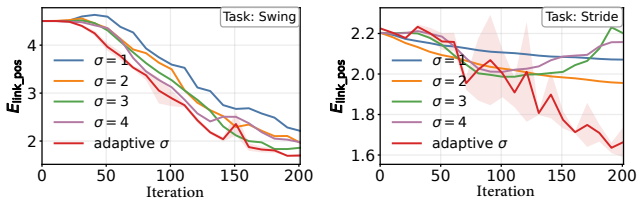


Figure 11: Comparison between adaptive and fixed perturbation intensities in real-world experiments on swing and stride motions.

To further evaluate FENCE's adaptive perturbation intensity mechanism, we trained three initial policies for the *swing* motion with different responsiveness to reference perturbations, primarily controlled by adjusting the tracking-related reward weight during PPO training. The corresponding results are shown in Figure 12. For weakly responsive policies (top), larger σ during data collection improves fine-tuning performance. In contrast, highly responsive policies (middle) require only smaller σ for sufficient exploration and optimal fine-tuning performance, whereas excessive perturbations degrade performance. For policies with intermediate responsiveness (bottom), moderate perturbations yield the best fine-tuning results. These findings indicate that the optimal perturbation intensity depends on the initial policy's responsiveness to reference perturbations. FENCE automatically adapts the perturbation intensity to different policies using a single set of hyperparameters, enabling efficient fine-tuning without manually tuning the perturbation scale for each policy.

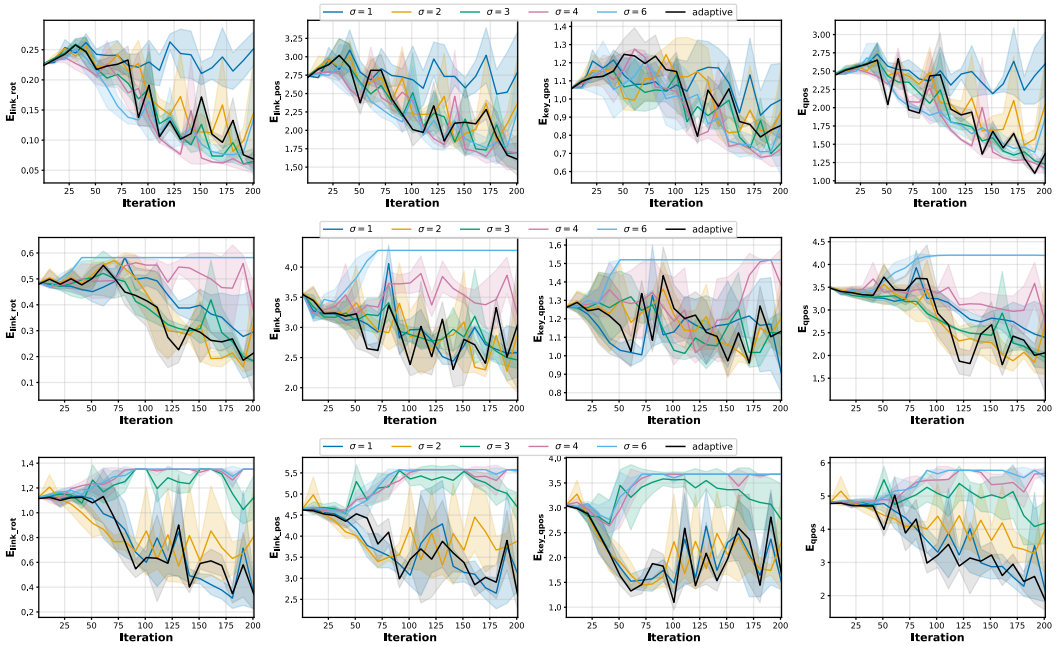


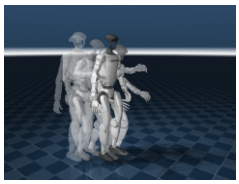
Figure 12: Comparison between the adaptive and fixed σ using policies with varying degrees of responsiveness (**top**: **weak**; **middle**: **medium**; **bottom**: **strong**).

G Applicability Beyond Ground-Contact Motions

The experiments in the main paper focus primarily on motions with continuous ground contact. To further evaluate the applicability of HumanoidNDM to motions involving flight phases, we extend our experiments to a jump motion, during which both feet leave the ground.

For motions with a flight phase, the root-height estimate h_t becomes unreliable due to the absence of valid foot contact. We therefore remove root height from the NDM input, output, and prediction loss, and model dynamics solely from proprioceptive history. For the NDM link-position loss and policy fine-tuning loss, global link positions are computed using the reference root height, avoiding the propagation of root-height estimation errors. All other components remain unchanged.

The jump motion and disturbance settings are described below.



Jump Starting from the default pose, the robot performs a jump motion during which both feet leave the ground. It then lands and returns to the default pose.

- *Extra Weight:* The mass of the `right_knee_link` is multiplied by 1.5, and its local center of mass is offset by [0, -0.2, 0].
- *Scaled KP:* The KP of the `left_hip_pitch` is decreased from 100 to 20.
- *Small Friction:* The coefficient of friction between the robot's feet and the ground is reduced from 1.0 to 0.3.

We evaluate HumanoidNDM under the same three types of dynamics disturbances used in the main simulation experiments. The quantitative results are reported in Table 7.

Table 7: Performance comparison of HumanoidNDM and baseline methods on the jump motion across 5 random trials.

Disturbance		Extra Weight				Scaled KP				Small Friction			
Trajectory	Method	$E_{\text{link_gpos}}$	$E_{\text{link_rot}}$	E_{gpos}	$E_{\text{key_gpos}}$	$E_{\text{link_gpos}}$	$E_{\text{link_rot}}$	E_{gpos}	$E_{\text{key_gpos}}$	$E_{\text{link_gpos}}$	$E_{\text{link_rot}}$	E_{gpos}	$E_{\text{key_gpos}}$
Jump	PPO w/ Disturb.	3.03	0.34	1.90	1.64	4.15	0.50	2.86	3.63	4.42	0.31	2.05	1.77
	Direct Transfer	6.74±0.00	1.56±0.00	4.77±0.00	6.50±0.00	7.41±0.00	1.77±0.00	5.01±0.00	8.75±0.00	7.53±0.00	1.80±0.00	4.92±0.00	6.74±0.00
	ASAP	**3.18±0.11**	0.63±0.07	3.30±0.14	3.15±0.20	7.08±0.09	1.98±0.12	4.56±0.16	5.98±0.41	4.59±0.09	0.47±0.03	3.52±0.02	5.05±0.27
	RWM	5.79±0.99	0.92±0.33	3.49±0.04	3.29±0.33	5.66±0.03	0.75±0.02	3.99±0.01	5.81±0.25	4.73±0.04	0.64±0.01	3.73±0.03	3.60±0.03
	HumanoidNDM	3.31±0.12	**0.37±0.01**	**2.97±0.03**	**2.42±0.09**	**4.68±0.22**	**0.55±0.03**	**3.25±0.05**	**5.78±0.34**	**4.28±0.98**	**0.46±0.19**	**2.63±0.28**	**2.40±0.68**

The fine-tuning curves are shown in Figure 13. HumanoidNDM consistently improves tracking performance under all three disturbances, demonstrating its applicability beyond motions with continuous ground contact.

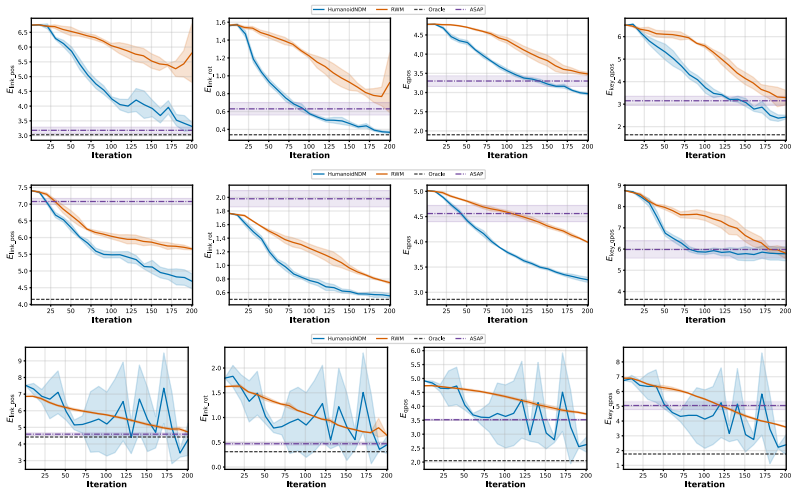


Figure 13: Jump: Extra Weight (top), Scaled KP (middle), Small Friction (bottom).

H Effectiveness of NDM Pretraining

We compare NDM with and without pretraining on domain-randomized simulation data in real-world experiments. Figure 14 shows that pretraining offers no improvement. We attribute this to the structured dynamics shifts brought by real-world disturbance, which are hard to cover by simulator.

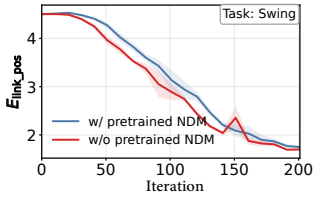
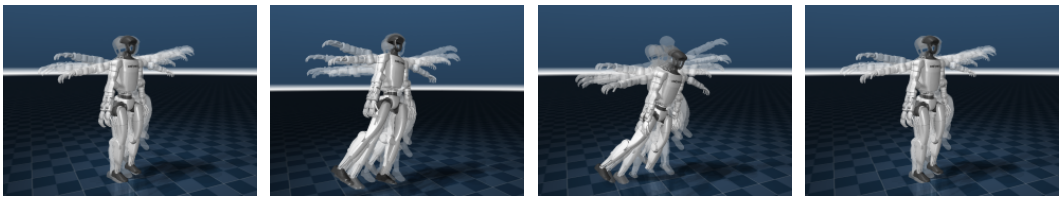


Figure 14: Comparison of HumanoidNDM with and without pretrained NDM.

I Visualization of Model Ablations

We compare three kinds of NDM: original implementation, a variant without history, and one without the Euclidean-space objective. Figures 15–19 demonstrate that both history information and the dual-space objective are crucial for rollout accuracy and policy fine-tuning performance.



(a) HumanoidNDM (b) w/o history (c) w/o Euclid objective (d) Ground Truth

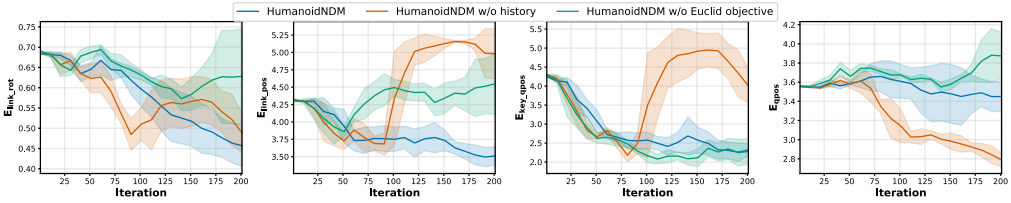
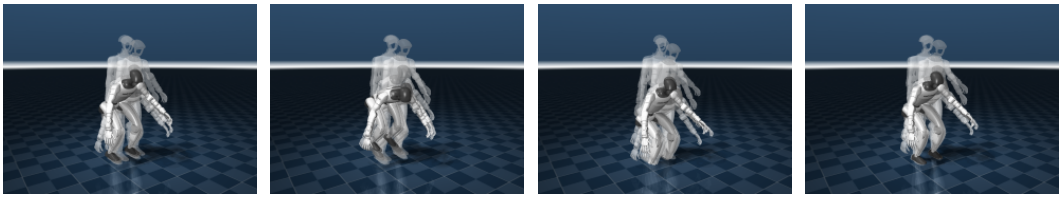


Figure 15: Model visualization and performance for balance motion with extra weight.



(a) HumanoidNDM (b) w/o history (c) w/o Euclid objective (d) Ground Truth

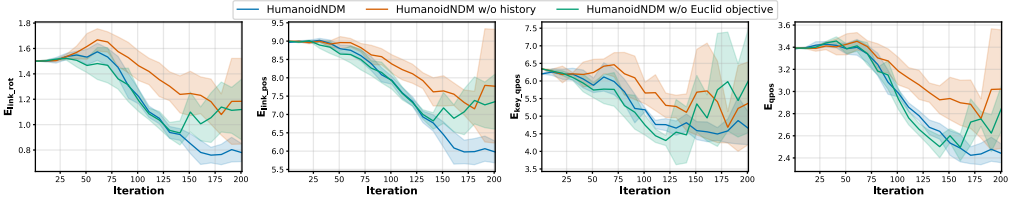
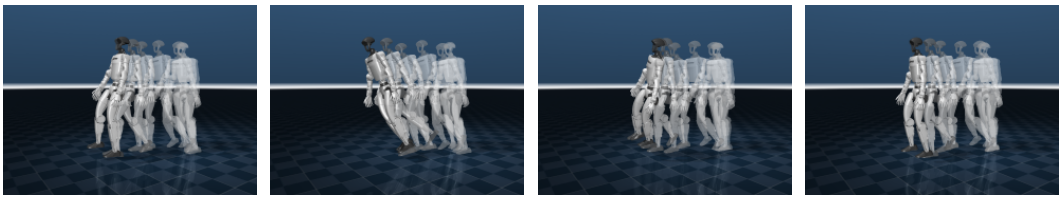


Figure 16: Model visualization and performance for squat motion with extra weight.



(a) HumanoidNDM (b) w/o history (c) w/o Euclid objective (d) Ground Truth

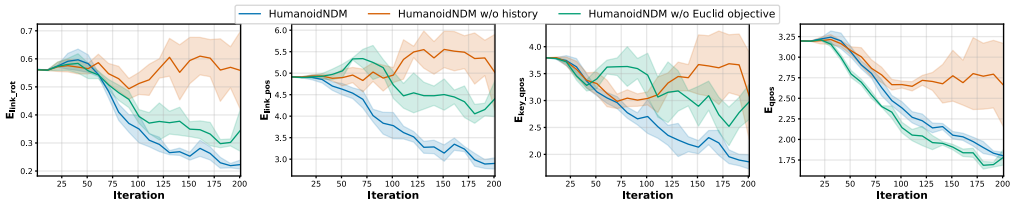
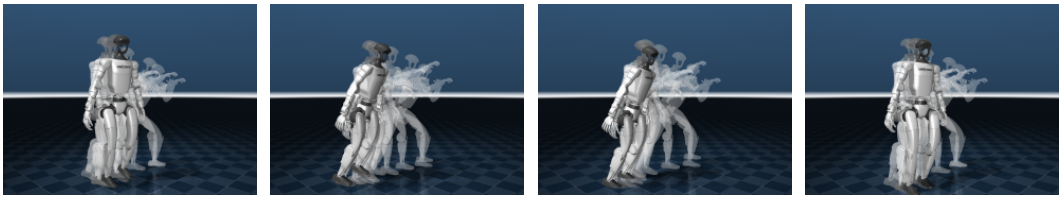


Figure 17: Model visualization and performance for walk motion with scaled KP.



(a) HumanoidNDM (b) w/o history (c) w/o Euclid objective (d) Ground Truth

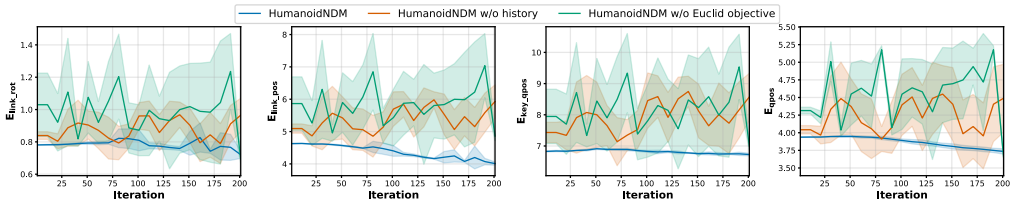
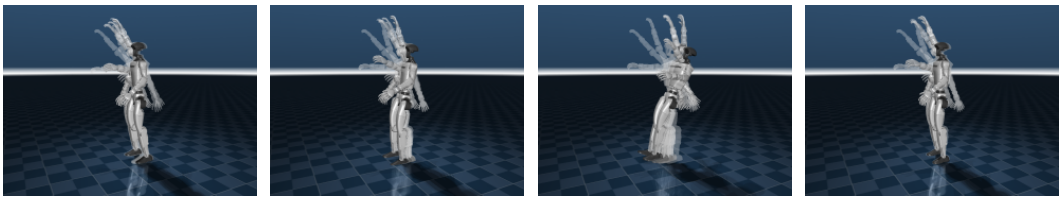


Figure 18: Model visualization and performance for stride motion with small friction.



(a) HumanoidNDM (b) w/o history (c) w/o Euclid objective (d) Ground Truth

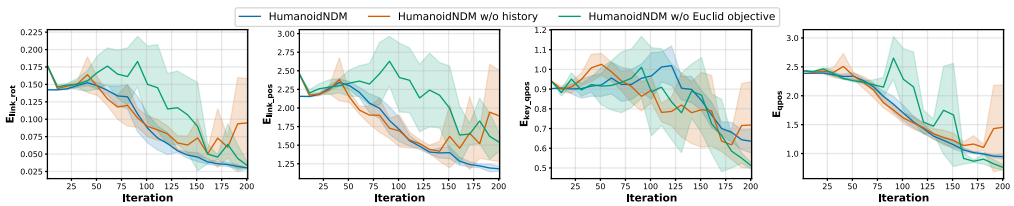


Figure 19: Model visualization and performance for swing motion with small friction.